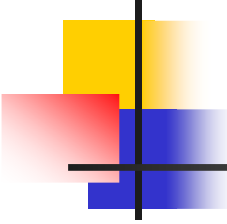


Hashing



Overview

- *Hash["string key"] ==> integer value*
- Hash Table Data Structure : Use-case
 - *To support insertion, deletion and search in average-case constant time*
 - Assumption: Order of elements irrelevant
 - ==> data structure not useful for sorting
- Hash table ADT
 - Implementations
 - Analysis
 - Applications

Hash Table

- One approach
 - Hash table is an array of fixed size TableSize
 - Array elements indexed by a key, which is mapped to an array index (0...TableSize-1)
 - Mapping (hash function) h from key to index
 - E.g., $h(\text{"john"}) = 3$

0	
1	
2	
3	john 25000
4	phil 31250
5	
6	dave 27500
7	mary 28200
8	
9	

Hash Table

- Insert

- $T[h(\text{"john"})] = T[3] = \langle \text{"john"}, 25000 \rangle$

- Delete

- $T[h(\text{"john"})] = \text{NULL}$

- Search

- Return $T[h(\text{"john"})]$

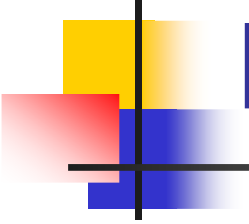
- What if $h(\text{"john"}) = h(\text{"joe"})$?

Hash
function

Hash key

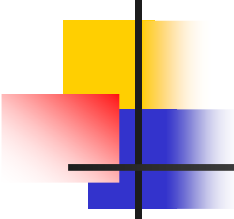
Data
record

0	
1	
2	
3	john 25000
4	phil 31250
5	
6	dave 27500
7	mary 28200
8	
9	



Factors affecting Hash Table Design

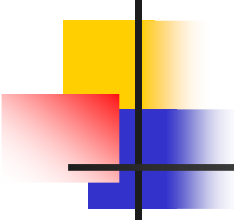
- Hash function
- Table size
 - Usually fixed at the start
- Collision handling scheme



Hash Function

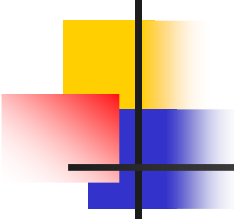
$h(\text{"key"}) \Rightarrow \text{hash table index}$

- Mapping from key to array index is called a hash function
 - Typically, many-to-one mapping
 - Different keys map to different indices
 - Should ideally distribute keys evenly over table
- Collision occurs when hash function maps two keys to same array index



Hash Function - Effective use of table size

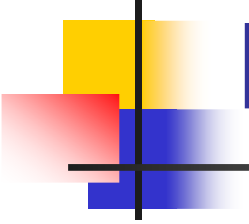
- Simple hash function (assume integer keys)
 - $h(\text{Key}) = \text{Key} \bmod \text{TableSize}$
- For random keys, $h()$ distributes keys evenly over table
- What if $\text{TableSize} = 100$ and keys are multiples of 10?
- Better if TableSize is a prime number
 - Not too close to powers of 2 or 10



Hash Function for String Keys

- Approach 1

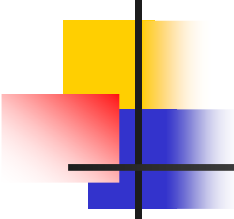
- Add up character ASCII values (0-127) to produce integer keys
 - E.g., "abcd" = $97+98+99+100 = 394$
 - $\Rightarrow h(\text{"abcd"}) = 394 \% \text{TableSize}$
- Small strings may not use all of table
 - $\text{Strlen}(S) * 127 < \text{TableSize}$
- Anagrams will map to the same index
 - $h(\text{"abcd"}) == h(\text{"dbac"})$



Hash Function for String Keys

- Approach 2

- Treat first 3 characters of string as base-27 integer (26 letters plus space)
- $\text{Key} = S[0] + (27 * S[1]) + (27^2 * S[2])$
- Assumes first 3 characters randomly distributed
 - Not true of English



Hash Function for String Keys

- Approach 3

- Use all N characters of string as an N-digit base-K integer

- Choose K to be prime number larger than number of different digits (characters)

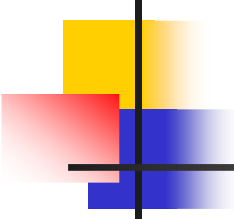
- I.e., K = 29, 31, 37

- If L = length of string S, then

$$h(S) = \left[\sum_{i=0}^{L-1} S[L-i-1] * 37^i \right] \bmod TableSize$$

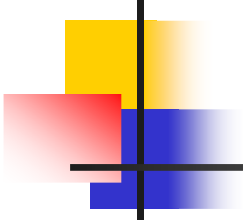
- Use Horner's rule to compute h(S)
- Limit L for long strings

```
1  /**
2   * A hash routine for string objects.
3   */
4  int hash( const string & key, int tableSize )
5  {
6      int hashVal = 0;
7
8      for( int i = 0; i < key.length( ); i++ )
9          hashVal = 37 * hashVal + key[ i ];
10
11     hashVal %= tableSize;
12     if( hashVal < 0 )
13         hashVal += tableSize;
14
15     return hashVal;
16 }
```



Collision Resolution

- What happens when $h(k_1) = h(k_2)$?
 - $\Rightarrow$ collision !
- Collision resolution strategies
 - *Chaining*
 - Store colliding keys in a linked list at the same hash table index
 - *Open addressing*
 - Store colliding keys elsewhere in the table



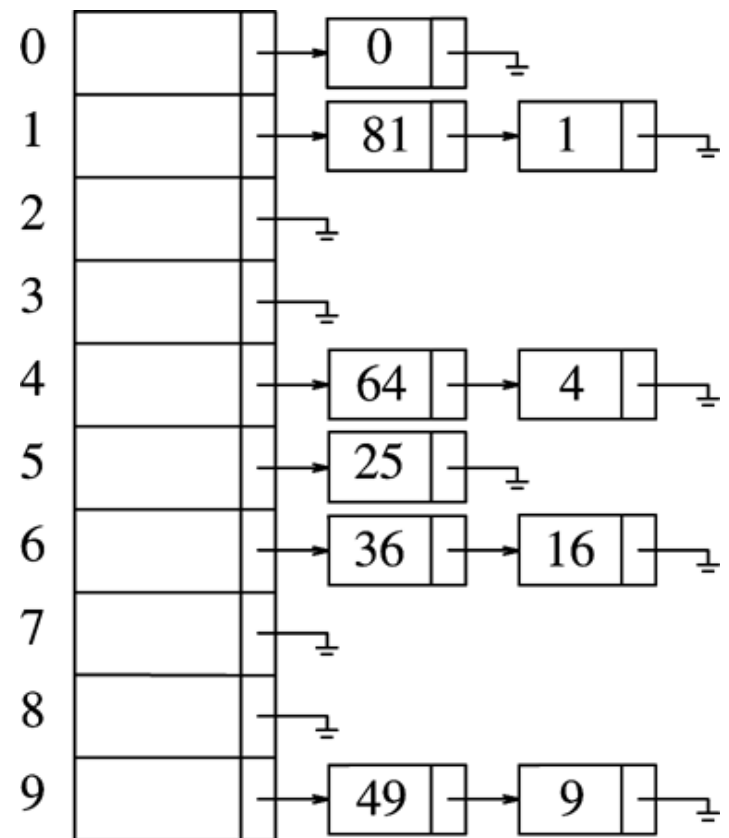
Chaining

Collision resolution approach #1

Collision Resolution by Chaining

- Hash table T is a vector of linked lists
 - Only singly-linked lists needed if memory is tight
- Key k is stored in list at $T[h(k)]$
- E.g., TableSize = 10
 - $h(k) = k \bmod 10$
 - Insert first 10 perfect squares

Insertion sequence: { 0 1 4 9 16 25 36 49 64 81 }

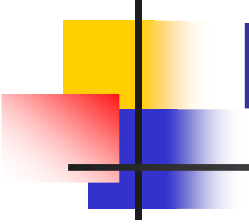


Implementation of Chaining Hash Table

```
1  template <typename HashedObj>
2  class HashTable
3  {
4      public:
5          explicit HashTable( int size = 101 );
6
7          bool contains( const HashedObj & x ) const;
8
9          void makeEmpty( );
10         void insert( const HashedObj & x );
11         void remove( const HashedObj & x );
12
13     private:
14         vector<list<HashedObj> > theLists;    // The array of Lists
15         int  currentSize;
16
17         void rehash( );
18         int myhash( const HashedObj & x ) const;
19     };
20
21     int hash( const string & key );
22     int hash( int key );
```

Vector of linked lists

Generic hash
functions for
integers and keys



Implementation of Chaining Hash Table

```
1      int myhash( const HashedObj & x ) const
2      {
3          int hashVal = hash( x );
4
5          hashVal %= theLists.size( );
6          if( hashVal < 0 )
7              hashVal += theLists.size( );
8
9          return hashVal;
10     }
```

```
1  bool insert( const HashedObj & x )
2  {
3      list<HashedObj> & whichList = theLists[ myhash( x ) ];
4      if( find( whichList.begin( ), whichList.end( ), x ) != whichList.end( ) )
5          return false;
6      whichList.push_back( x );
7
8          // Rehash; see Section 5.5
9      if( ++currentSize > theLists.size( ) )
10         rehash( );
11
12     return true;
13 }
```

Duplicate check

Later, but essentially
resizes the hashtable if its
getting crowded

```

1  void makeEmpty( )
2  {
3      for( int i = 0; i < theLists.size( ); i++ )
4          theLists[ i ].clear( );
5  }
6
7  bool contains( const HashedObj & x ) const
8  {
9      const list<HashedObj> & whichList = theLists[ myhash( x ) ];
10     return find( whichList.begin( ), whichList.end( ), x ) != whichList.end( );
11 }
12
13 bool remove( const HashedObj & x )
14 {
15     list<HashedObj> & whichList = theLists[ myhash( x ) ];
16     list<HashedObj>::iterator itr = find( whichList.begin( ), whichList.end( ), x );
17
18     if( itr == whichList.end( ) )
19         return false;
20
21     whichList.erase( itr );
22     --currentSize;
23     return true;
24 }

```

Each of these operations takes time linear in the length of the list, at the hashed index location

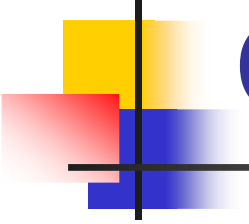
```

1  // Example of an Employee class
2  class Employee
3  {
4      public:
5          const string & getName( ) const
6              { return name; }
7
8          bool operator==( const Employee & rhs ) const
9              { return getName( ) == rhs.getName( ); }
10         bool operator!=( const Employee & rhs ) const
11             { return !( *this == rhs; }
12
13         // Additional public members not shown
14
15     private:
16         string name;
17         double salary;
18         int    seniority;
19
20         // Additional private members not shown
21 };
22
23 int hash( const Employee & item )
24 {
25     return hash( item.getName( ) );
26 }

```

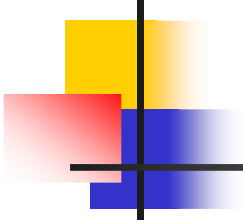
All hash objects must define == and != operators.

Hash function to handle Employee object type



Collision Resolution by Chaining: Analysis

- Load factor λ of a hash table T
 - N = number of elements in T
 - M = size of T
 - $\lambda = N/M$
 - I.e., λ is the average length of a chain
- Unsuccessful search $O(\lambda)$
- Successful search $O(\lambda/2)$
- Ideally, want $\lambda \leq 1$ (not a function of N)

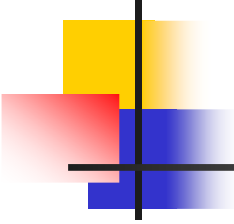


Open Addressing

Collision resolution approach #2

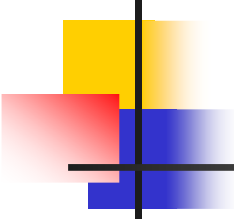
Look, Open addressing is almost like chaining in that you can consider the probe sequence to be a chain, the only difference is that, some entries in the probe sequence can already be a hash value of another string, then we will need to search further... So load factor needs to be almost 0.5 for good performance

Collision Resolution by Open Addressing



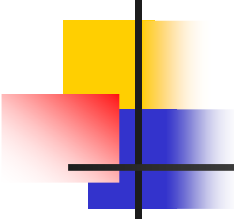
When a collision occurs, look elsewhere in the table for an empty slot

- Advantages over chaining
 - No need for list structures
 - No need to allocate/deallocate memory during insertion/deletion (slow)
- Disadvantages
 - Slower insertion – May need several attempts to find an empty slot
 - Table needs to be bigger (than chaining-based table) to achieve average-case constant-time performance
 - Load factor $\lambda \approx 0.5$



Collision Resolution by Open Addressing

- Probe sequence
 - Sequence of slots in hash table to search
 - $h_0(x), h_1(x), h_2(x), \dots$
 - Needs to visit each slot exactly once
 - Needs to be repeatable (so we can find/delete what we've inserted)
- Hash function
 - $h_i(x) = (h(x) + f(i)) \bmod \text{TableSize}$
 - $f(0) = 0 \qquad \qquad \qquad ==> \text{first try}$



Linear Probing

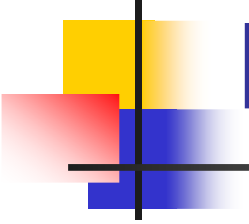
- $f(i)$ is a linear function of i
 - $f(i) = i$
 - Probe sequence: $+0, +1, +2, +3, +4, \dots$
- Example: $h(x) = x \bmod \text{TableSize}$
 - $h_0(89) = (h(89) + f(0)) \bmod 10 = 9$
 - $h_0(18) = (h(18) + f(0)) \bmod 10 = 8$
 - $h_0(49) = (h(49) + f(0)) \bmod 10 = 9$ (X)
 - $h_1(49) = (h(49) + f(1)) \bmod 10$
 $= (h(49) + 1) \bmod 10 = 0$

Linear Probing Example

Insert sequence: 89, 18, 49, 58, 68

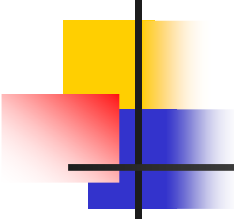
	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1					58	58
2						69
3						
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

—————→ time



Linear Probing: Issues

- Probe sequences can get long
- Primary clustering
 - Keys tend to cluster in one part of table
 - Keys that hash into cluster will be added to the end of the cluster (making it even bigger)



Linear Probing: Analysis

- Expected number of probes for insertion or unsuccessful search

$$\frac{1}{2} \left(1 + \frac{1}{(1-\lambda)^2} \right)$$

- Expected number of probes for successful search

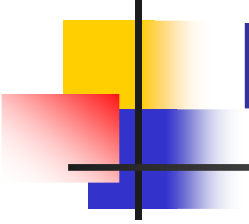
$$\frac{1}{2} \left(1 + \frac{1}{(1-\lambda)} \right)$$

- Example ($\lambda = 0.5$)

- Insert / unsuccessful search
 - 2.5 probes
- Successful search
 - 1.5 probes

- Example ($\lambda = 0.9$)

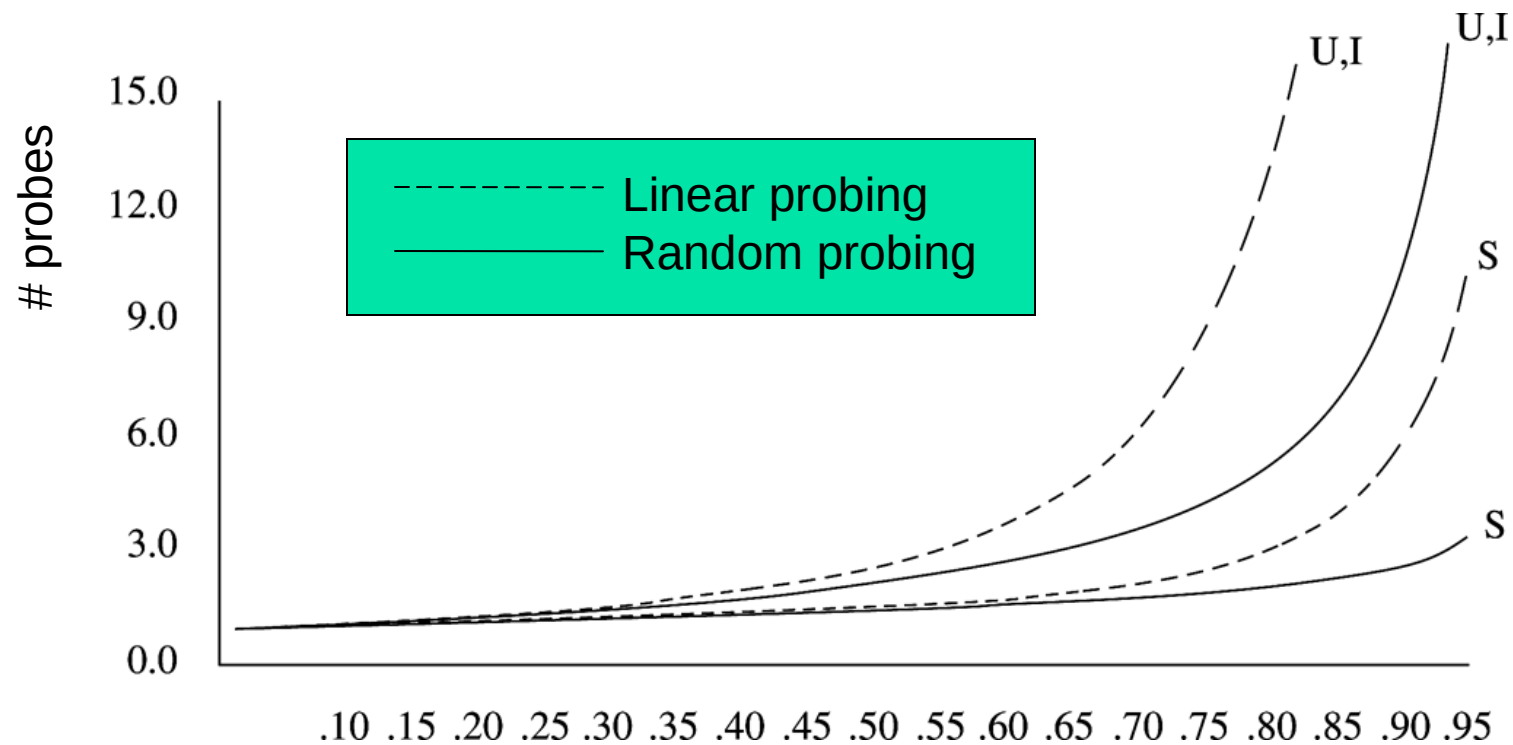
- Insert / unsuccessful search
 - 50.5 probes
- Successful search
 - 5.5 probes



Random Probing: Analysis

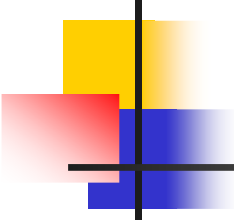
- Random probing does not suffer from clustering
- Expected number of probes for insertion or unsuccessful search: $\frac{1}{\lambda} \ln \frac{1}{1-\lambda}$
- Example
 - $\lambda = 0.5$: 1.4 probes
 - $\lambda = 0.9$: 2.6 probes

Linear vs. Random Probing



U - unsuccessful search
S - successful search
I - insert

Load factor λ



Quadratic Probing

- Avoids primary clustering
- $f(i)$ is quadratic in i
 - $f(i) = i^2$
 - Probe sequence: $+0, +1, +4, +9, +16, \dots$
- Example
 - $h_0(58) = (h(58) + f(0)) \bmod 10 = 8$ (X)
 - $h_1(58) = (h(58) + f(1)) \bmod 10 = 9$ (X)
 - $h_2(58) = (h(58) + f(2)) \bmod 10 = 2$

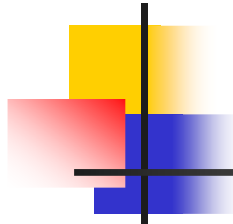
Quadratic Probing Example

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49 #1	49	49 #1
1						
2					58 #2	58
3						69 #2
4						
5						
6						
7						
8			18 #0	18	18 #0	18
9		89 #0	89	89 #0	89 #1	89 #0

Q Delete(49), Find(69) - is there a problem?

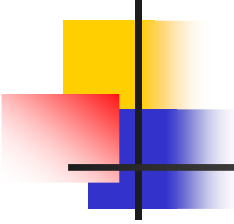
Problem with eager deletion:

Say, 49 is deleted here, so that slot will become empty, now if we want to find 69, we will look at the slot where 49 previously was, and we will see that his slot is empty, then we will actually decide that 69 doesn't exist in the hashtable, which is wrong. Thus we just add a marker for 49 as deleted.



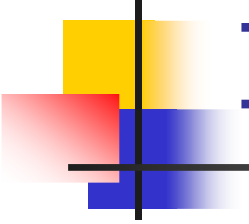
Quadratic Probing: Analysis

- Difficult to analyze
- Theorem 5.1
 - New element can always be inserted into a table that is at least half empty and TableSize is prime
- Otherwise, may never find an empty slot, even if one exists
- Ensure table never gets half full
 - If close, then expand it



Quadratic Probing

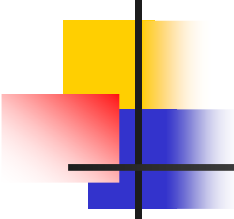
- May cause “secondary clustering”
- Deletion
 - Emptying slots can break probe sequence and could cause find stop prematurely
 - Lazy deletion
 - Differentiate between empty and deleted slot
 - Skip deleted slots
 - Slows operations (effectively increases λ)



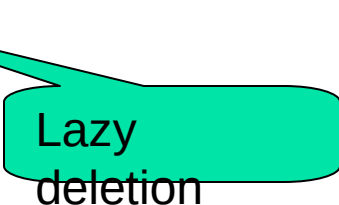
Quadratic Probing: Implementation

```
1  template <typename HashedObj>
2  class HashTable
3  {
4      public:
5          explicit HashTable( int size = 101 );
6
7          bool contains( const HashedObj & x ) const;
8
9          void makeEmpty( );
10         bool insert( const HashedObj & x );
11         bool remove( const HashedObj & x );
12
```

Quadratic Probing: Implementation



```
13     enum EntryType { ACTIVE, EMPTY, DELETED };
14
15     private:
16         struct HashEntry
17         {
18             HashedObj element;
19             EntryType info;
20
21             HashEntry( const HashedObj & e = HashedObj( ), EntryType i = EMPTY )
22                 : element( e ), info( i ) { }
23         };
24
25         vector<HashEntry> array;
26         int currentSize;
27
28         bool isActive( int currentPos ) const;
29         int findPos( const HashedObj & x ) const;
30         void rehash( );
31         int myhash( const HashedObj & x ) const;
32     };
```



Lazy
deletion

Quadratic Probing: Implementation

```
1    explicit HashTable( int size = 101 ) : array( nextPrime( size ) )
2        { makeEmpty( ); }
3
4    void makeEmpty( )
5    {
6        currentSize = 0;
7        for( int i = 0; i < array.size( ); i++ )
8            array[ i ].info = EMPTY;
9    }
```

Ensure table
size is prime

Quadratic Probing: Implementation

```
1  bool contains( const HashedObj & x ) const
2      { return isActive( findPos( x ) ); }
3
4  int findPos( const HashedObj & x ) const
5  {
6      int offset = 1;
7      int currentPos = myhash( x );
8
9      while( array[ currentPos ].info != EMPTY &&
10             array[ currentPos ].element != x )
11      {
12          currentPos += offset; // Compute ith probe
13          offset += 2;
14          if( currentPos >= array.size( ) )
15              currentPos -= array.size( );
16      }
17
18      return currentPos;
19  }
20
21  bool isActive( int currentPos ) const
22      { return array[ currentPos ].info == ACTIVE; }
```

Find

Skip
DELETED;
No duplicates

Quadratic probe
sequence (really)

Quadratic Probing: Implementation

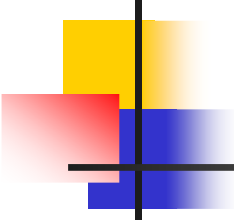
```
1  bool insert( const HashedObj & x )
2  {
3      // Insert x as active
4      int currentPos = findPos( x );
5      if( isActive( currentPos ) )
6          return false;
7
8      array[ currentPos ] = HashEntry( x, ACTIVE );
9
10     // Rehash; see Section 5.5
11     if( ++currentSize > array.size( ) / 2 )
12         rehash( );
13
14     return true;
15 }
16
17 bool remove( const HashedObj & x )
18 {
19     int currentPos = findPos( x );
20     if( !isActive( currentPos ) )
21         return false;
22
23     array[ currentPos ].info = DELETED;
24     return true;
25 }
```

Insert

No duplicates

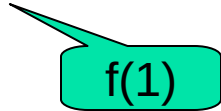
Remove

No
deallocation
needed

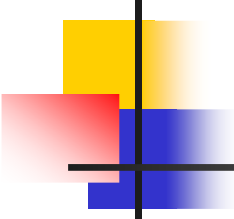


Double Hashing

- Use a second hash function for tries
 - $f(i) = i * h_2(x)$
- Good choices for $h_2(x)$?
 - Should never evaluate to 0
 - $h_2(x) = R - (x \bmod R)$
 - R is prime number less than TableSize
- Previous example with $R=7$
 - $h_0(49) = (h(49) + f(0)) \bmod 10 = 9$ (X)
 - $h_1(49) = (h(49) + 1 * (7 - 49 \bmod 7)) \bmod 10 = 6$

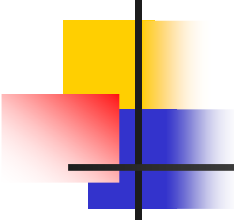


f(1)



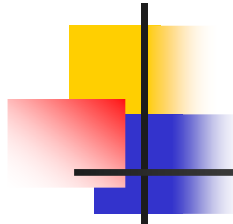
Double Hashing Example

	Empty Table	After 89	After 18	After 49	After 58	After 69
0						69
1						
2						
3					58	58
4						
5						
6				49	49	49
7						
8			18	18	18	18
9		89	89	89	89	89



Double Hashing: Analysis

- Imperative that TableSize is prime
 - E.g., insert 23 into previous table
- Empirical tests show double hashing close to random hashing
- Extra hash function takes extra time to compute



Rehashing

- Increases the size of the hash table when load factor too high
- Typically expand the table to twice its size (but still prime)
- Need to reinsert all existing elements into new hash table

Rehashing Example

$$h(x) = x \bmod 7$$
$$\lambda = 0.57$$

0	6
1	15
2	
3	24
4	
5	
6	13

Insert 23

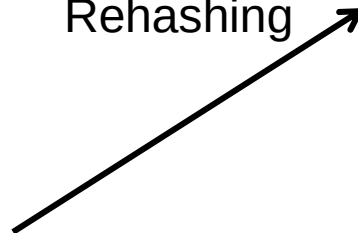


$$\lambda = 0.71$$

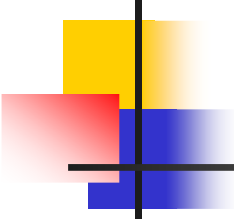
0	6
1	15
2	23
3	24
4	
5	
6	13

$$h(x) = x \bmod 17$$
$$\lambda = 0.29$$

Rehashing

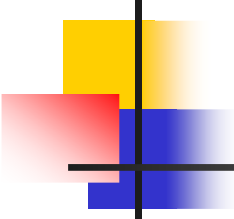


0	
1	
2	
3	
4	
5	
6	6
7	23
8	24
9	
10	
11	
12	
13	13
14	
15	15
16	



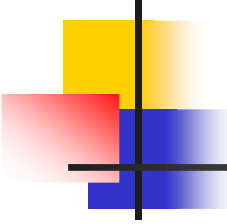
Rehashing Analysis

- Rehashing takes $O(N)$ time
- But happens infrequently
- Specifically
 - Must have been $N/2$ insertions since last rehash
 - Amortizing the $O(N)$ cost over the $N/2$ prior insertions yields only constant additional time per insertion



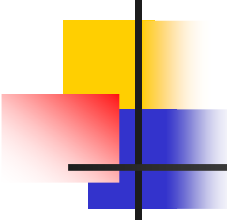
Rehashing Implementation

- When to rehash
 - When table is half full ($\lambda = 0.5$)
 - When an insertion fails
 - When load factor reaches some threshold
- Works for chaining and open addressing



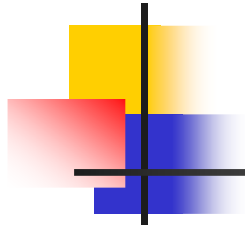
Rehashing for Chaining

```
20      /**
21       * Rehashing for separate chaining hash table.
22       */
23      void rehash( )
24      {
25          vector<list<HashedObj> > oldLists = theLists;
26
27          // Create new double-sized, empty table
28          theLists.resize( nextPrime( 2 * theLists.size( ) ) );
29          for( int j = 0; j < theLists.size( ); j++ )
30              theLists[ j ].clear( );
31
32          // Copy table over
33          currentSize = 0;
34          for( int i = 0; i < oldLists.size( ); i++ )
35          {
36              list<HashedObj>::iterator itr = oldLists[ i ].begin( );
37              while( itr != oldLists[ i ].end( ) )
38                  insert( *itr++ );
39          }
40      }
```



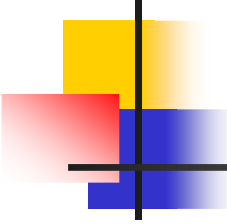
Rehashing for Quadratic Probing

```
1      /**
2      * Rehashing for quadratic probing hash table.
3      */
4      void rehash( )
5      {
6          vector<HashEntry> oldArray = array;
7
8          // Create new double-sized, empty table
9          array.resize( nextPrime( 2 * oldArray.size( ) ) );
10         for( int j = 0; j < array.size( ); j++ )
11             array[ j ].info = EMPTY;
12
13         // Copy table over
14         currentSize = 0;
15         for( int i = 0; i < oldArray.size( ); i++ )
16             if( oldArray[ i ].info == ACTIVE )
17                 insert( oldArray[ i ].element );
18     }
```



Hash Tables in C++ STL

- Hash tables not part of the C++ Standard Library
- Some implementations of STL have hash tables (e.g., SGI's STL)
 - `hash_set`
 - `hash_map`



Hash Set in SGI's STL

```
#include <hash_set>

struct eqstr
{
    bool operator()(const char* s1, const char* s2) const
    {
        return strcmp(s1, s2) == 0;
    }
};

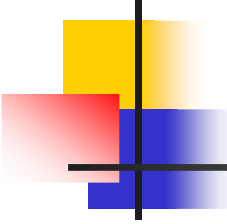
void lookup(const hash_set<const char*, hash<const char*>, eqstr>& Set,
            const char* word)
{
    hash_set<const char*, hash<const char*>, eqstr>::const_iterator it
        = Set.find(word);
    cout << word << ": "
         << (it != Set.end() ? "present" : "not present")
         << endl;
}

int main()
{
    hash_set<const char*, hash<const char*>, eqstr> Set;
    Set.insert("kiwi");
    lookup(Set, "kiwi");
}
```

Key

Hash fn

Key equality
test



Hash Map in SGI's STL

```
#include <hash_map>
```

```
struct eqstr
{
    bool operator() (const char* s1, const char* s2) const
    {
        return strcmp(s1, s2) == 0;
    }
};
```

```
int main()
{
    hash_map<const char*, int, hash<const char*>, eqstr> months;
    months["january"] = 31;
    months["february"] = 28;
    ...
    months["december"] = 31;
    cout << "january -> " << months["january"] << endl;
}
```

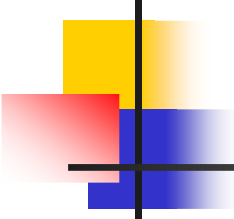
Key

Data

Hash fn

Key equality

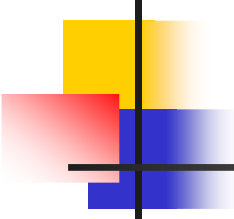
test



Problem with Large Tables

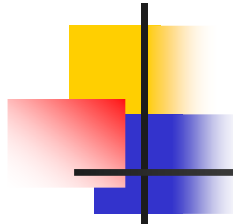
- What if hash table is too large to store in main memory?
- Solution: Store hash table on disk
 - Minimize disk accesses
- But...
 - Collisions require disk accesses
 - Rehashing requires a lot of disk accesses

Solution: **Extendible Hashing**



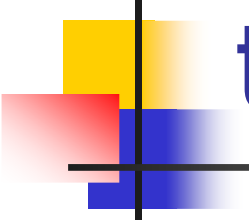
Hash Table Applications

- Symbol table in compilers
- Accessing tree or graph nodes by name
 - E.g., city names in Google maps
- Maintaining a transposition table in games
 - Remember previous game situations and the move taken (avoid re-computation)
- Dictionary lookups
 - Spelling checkers
 - Natural language understanding (word sense)



Summary

- Hash tables support fast insert and search
 - $O(1)$ average case performance
 - Deletion possible, but degrades performance
- Not good if need to maintain ordering over elements
- Many applications



Points to remember - Hash tables

- Table size prime
- Table size much larger than number of inputs (to maintain λ closer to 0 or < 0.5)
- Tradeoffs between chaining vs. probing
- Collision chances decrease in this order: linear probing $\Rightarrow$ quadratic probing $\Rightarrow$ {random probing, double hashing}
- Rehashing required to resize hash table at a time when λ exceeds 0.5
- Good for searching. Not good if there is some order implied by data.